



Bericht

Entwicklung einer Metaheuristik für das Job Shop Scheduling Problem unter Berücksichtigung von reihenfolgeabhängigen Rüstzeiten

Jakob Weber Matr.-Nr. 3869088

Johannes Walz Matr.-Nr. 3533824

Prüfer: Prof.Dr.David Müller

Inhaltsverzeichnis

1	Einleitung	2
1.1	Problemstellung	2
1.2	Zielsetzung	2
1.3	Aufbau der Arbeit	3
2	Notation	4
3	Methodik	7
3.1	Eröffnungsheuristik - GT-Algorithmus	7
3.2	Metaheuristik - Tabu-Suche	11
3.2.1	Kritischer Pfad und kritische Blöcke	12
3.2.2	Nachbarschaftsdefinitionen	14
3.3	Hard- und Softwarespezifikationen	15
4	Ergebnisse	16
4.1	Eröffnungsheuristik Ergebnisse	16
4.2	Metaheuristik Ergebnisse	18
4.3	Vergleich CP-Solver	23

Kapitel 1

Einleitung

1.1 Problemstellung

Die Planung von Produktionsabläufen ist eine zentrale Aufgabe in der Fertigungssteuerung. Unternehmen müssen verfügbare Maschinen, Arbeitsplätze und Zeiträume so einsetzen, dass Aufträge effizient bearbeitet werden können. Dabei entstehen Abhängigkeiten zwischen einzelnen Arbeitsschritten. Operationen eines Auftrags müssen in einer festgelegten Reihenfolge ausgeführt werden. Gleichzeitig konkurrieren verschiedene Aufträge um dieselben Maschinen. Solche Planungsprobleme treten unter anderem in der industriellen Fertigung, der Montage, der Instandhaltung und der Auftragsproduktion auf. Die Qualität eines Produktionsplans beeinflusst die Durchlaufzeiten, die Maschinenauslastung und die Einhaltung von Lieferterminen. Bereits kleine Änderungen der Auftragsreihenfolge können den gesamten Produktionsablauf verändern. Eine zusätzliche Herausforderung entsteht durch Rüstvorgänge. Maschinen müssen häufig angepasst, gereinigt oder mit anderen Werkzeugen ausgestattet werden, bevor die nächste Operation ausgeführt werden kann. Die Dauer dieser Umrüstung hängt dabei von der Reihenfolge der bearbeiteten Aufträge ab. Eine ungeeignete Reihenfolge führt zu längeren Rüstzeiten und verlängert die gesamte Produktionsdauer.

1.2 Zielsetzung

Die primäre Zielsetzung dieser Arbeit ist die Entwicklung und Evaluierung einer Metaheuristik zur Lösung des Job-Shop-Scheduling-Problems mit sequenzabhängigen Rüstzeiten. Als algorithmischer Ansatz wird hierfür eine Tabu-Suche implementiert. Im Zentrum der Untersuchung steht die Beantwortung der Frage, wie sich die Wahl der Nachbarschaftsdefinition auf den resultierenden Makespan auswirkt. Dazu werden drei unterschiedliche Nachbarschaftsstrukturen systematisch analysiert und miteinander verglichen.

Konkret betrifft dies die Adjacent-Swap-, die N5- und die N6-Nachbarschaft. Ein wichtiges

Teilziel ist zudem die Implementierung einer geeigneten Eröffnungsheuristik. Hierfür wird der Giffler-Thompson-Algorithmus genutzt, um zulässige Startlösungen für das Suchverfahren bereitzustellen. Zur Validierung der Leistungsfähigkeit wird die Tabu-Suche auf ein vorgegebenes Testset angewendet.

Die Ergebnisse werden anschließend einer quantitativen Analyse unterzogen. Als Performance-Referenz dient dabei ein exakter CP-Solver auf Basis der Google OR-Tools. Ziel ist es, die Stärken und Schwächen der jeweiligen Nachbarschaften hinsichtlich der Lösungsqualität und der benötigten Rechenzeit systematisch offenzulegen.

1.3 Aufbau der Arbeit

Diese Arbeit gliedert sich in fünf Kapitel. Nach der Einleitung führt Kapitel 2 die mathematische Notation für das betrachtete Job-Shop-Scheduling-Problem ein. Diese dient als formale Grundlage für die Algorithmen und die zugehörige Bewertungslogik. Dabei werden die Symbole für das Kernproblem, die Eröffnungsheuristik und die Tabu-Suche definiert.

Kapitel 3 beschreibt die methodische Umsetzung der Lösungsverfahren. Zunächst wird der Giffler-Thompson-Algorithmus zur Erzeugung zulässiger Startlösungen vorgestellt.

Anschließend wird die Funktionsweise der Tabu-Suche dargelegt. Dies umfasst die Berechnung des kritischen Pfades, die Extraktion kritischer Blöcke sowie die Definition der Nachbarschaften Adjacent-Swap, N5 und N6. Den Abschluss bilden die Hard- und Softwarespezifikationen.

Kapitel 4 präsentiert die Ergebnisse der Evaluation. Hierbei wird zunächst die Leistung der verschiedenen Prioritätsregeln ausgewertet. Zudem werden die drei Nachbarschaftsdefinitionen quantitativ miteinander verglichen. Die Leistungsfähigkeit der Metaheuristik wird schließlich durch den Vergleich mit einem exakten CP-Solver validiert.

Kapitel 5 schließt die Arbeit mit einer kritischen Diskussion der Ergebnisse ab. Dabei werden die Stärken und Schwächen der jeweiligen Nachbarschaften hinsichtlich Lösungsqualität und Rechenzeit systematisch offengelegt.

Kapitel 2

Notation

Notation des Job-Shop-Scheduling-Problems mit sequenzabhängigen Rüstzeiten

In diesem Kapitel wird die Notation für das betrachtete Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten eingeführt. Die Notation dient als Grundlage für die Beschreibung der Tabu-Search-Heuristik und der verwendeten Bewertungslogik.

Gegeben ist eine Menge von Jobs J und eine Menge von Maschinen M . Jeder Job $j \in J$ besteht aus einer geordneten Folge von Operationen

$$O_j = (o_{j,1}, o_{j,2}, \dots, o_{j,q_j}).$$

Die Reihenfolge der Operationen innerhalb eines Jobs ist vorgegeben. Eine Operation $o_{j,k}$ darf daher erst beginnen, wenn ihre direkte Vorgängeroperation $o_{j,k-1}$ abgeschlossen ist.

Jede Operation $o_{j,k}$ wird auf genau einer Maschine $m_{j,k} \in M$ bearbeitet und besitzt eine Bearbeitungszeit $p_{j,k}$. Zwischen zwei Operationen, die auf derselben Maschine direkt aufeinanderfolgen, kann eine sequenzabhängige Rüstzeit $s_{u,v}$ anfallen. Diese hängt von der vorherigen Operation u und der nachfolgenden Operation v ab. Ziel des Algorithmus ist es, eine Reihenfolge der Operationen auf den Maschinen zu finden, die den Makespan $C_{\max}$ minimiert. Der Makespan ist definiert als der Zeitpunkt, zu dem die letzte Operation aller Jobs abgeschlossen ist.

Notation der Eröffnungsheuristik

Zur Erzeugung einer zulässigen Startlösung wird der Giffler-Thompson-Algorithmus verwendet. Für die mathematische Formulierung dieses Verfahrens werden folgende zusätzliche Sym-

bole und Hilfsvariablen benötigt:

Tabelle 1: Notation der Eröffnungsheuristik (Giffler-Thompson-Algorithmus)

Symbol	Bedeutung
U	Menge der noch unverplanten Operationen, $U \subseteq O$
L_m	aktuelle Last der Maschine $m \in M$
$\lambda(m)$	Index des zuletzt auf Maschine m eingeplanten Jobs
$r(o)$	job-basierter frühester Startzeitpunkt (Bereitschaftszeit) von Operation o
$r_m(o)$	frühestmöglicher Maschinen-Startzeitpunkt von Operation o
C^*	minimaler frühestmöglicher Endzeitpunkt der aktuellen Iteration
m^*	kritische Maschine, auf der C^* erreicht wird
K	Konfliktmenge einplanbarer Operationen auf Maschine m^*
π	angewendete Prioritätsregel

Notation der Tabu-Suche

Zur Lösung des Problems wird eine Tabu-Search-Heuristik verwendet. Die aktuelle Lösung wird mit S bezeichnet. Sie wird im Vorfeld durch den Giffler-Thompson-Algorithmus erzeugt. Die beste bisher gefundene Lösung wird als S^* notiert. Die Nachbarschaft der aktuellen Lösung wird durch $NB(S)$ beschrieben. Sie enthält alle Lösungen, die durch zulässige lokale Bewegungen aus S erzeugt werden können.

Die Tabu-Liste TL speichert Bewegungsattribute, die für eine begrenzte Anzahl von Iterationen tabu sind. Dadurch wird ein unmittelbares Zurückkehren in vorherige Lösungen verhindert. Tabuierte Bewegungen können durch das Aspirationskriterium zugelassen werden, wenn sie zu einer neuen globalen Bestlösung führen.

Tabelle 2: Notation der Tabu-Search-Heuristik

Symbol	Bedeutung
$\hat{S}$	erzeugte Startlösung
S	aktuelle Lösung
S^*	beste bisher gefundene Lösung
S'	Nachbarlösung der aktuellen Lösung
$NB(S)$	Nachbarschaft der aktuellen Lösung S
TL	Tabu-Liste
TL^{Dauer}	Gültigkeitsdauer eines Tabu-Eintrags
I	Anzahl der Iterationen ohne Verbesserung
I^{max}	maximale Anzahl an Iterationen ohne Verbesserung
I_{total}	Gesamtzahl der Iterationen
$I_{\text{total}}^{\text{max}}$	maximale Gesamtzahl an Iterationen
$I^{\text{Diversifikation}}$	Schwelle für zusätzliche Diversifikationsbewegungen
τ^{max}	optionales Zeitlimit
$F(S)$	Zielfunktionswert der Lösung S

Da der Makespan minimiert wird, gilt äquivalent:

$$C_{\max}(S') < C_{\max}(S^*).$$

Kapitel 3

Methodik

3.1 Eröffnungsheuristik - GT-Algorithmus

Algorithm 1 Giffler-Thompson-Algorithmus

```
1:  $U \leftarrow O$ 
2:  $L_m \leftarrow 0 \quad \forall m \in M$ 
3:  $\lambda(m) \leftarrow \emptyset \quad \forall m \in M$ 
4:  $r(o_{j,1}) \leftarrow 0 \quad \forall j \in J$ 
5:  $r(o_{j,k}) \leftarrow \infty \quad \forall j \in J, k > 1$ 
6: while  $U \neq \emptyset$  do
7:   Berechne für alle  $o_{j,k} \in U$  mit  $r(o_{j,k}) < \infty$  den frühesten Startzeitpunkt:
      $r_m(o_{j,k}) \leftarrow \max\{L_{m_{j,k}} + s_{\lambda(m_{j,k}),j}, r(o_{j,k})\}$ 
8:    $C^* \leftarrow \min\{r_m(o_{j,k}) + p_{j,k} \mid o_{j,k} \in U \text{ und } r(o_{j,k}) < \infty\}$ 
9:   Sei  $m^*$  die Maschine, auf der  $C^*$  erreicht wird
10:  Bilde Konfliktmenge  $K \leftarrow \{o_{j,k} \in U \mid m_{j,k} = m^*, r(o_{j,k}) < \infty, r_m(o_{j,k}) < C^*\}$ 
11:  Wähle Operation  $o' = o_{j',k'} \in K$  gemäß der Prioritätsregel  $\pi$ 
12:   $t(o') \leftarrow r_m(o')$ 
13:   $C(o') \leftarrow t(o') + p_{j',k'}$ 
14:   $L_{m^*} \leftarrow C(o')$ 
15:   $\lambda(m^*) \leftarrow j'$ 
16:  if  $k' < q_{j'}$  then
17:     $r(o_{j',k'+1}) \leftarrow C(o')$ 
18:  end if
19:   $U \leftarrow U \setminus \{o'\}$ 
20: end while
```

Der Giffler-Thompson-Algorithmus [1] wird als Eröffnungsheuristik zur Generierung einer zulässigen Startlösung für das Job Shop Scheduling Problem verwendet.

Der Algorithmus beginnt mit der Initialisierung der Menge U aller unverplanten Operationen, die zu Beginn der Gesamtmenge O entspricht. Die Maschinenlasten L_m sowie die Historie der zuletzt auf einer Maschine eingeplanten Jobs $\lambda(m)$ werden für alle Maschinen $m \in M$ auf Null bzw. die leere Menge $\emptyset$ gesetzt. Die Bereitschaftszeit $r(o_{j,k})$ wird für die jeweils ersten Operationen aller Jobs auf Null und für alle nachfolgenden Operationen auf unendlich initialisiert, um zu signalisieren, dass diese noch nicht einplanbar sind.

In jeder Iteration der Hauptschleife wird für alle einplanbaren Operationen in U der frühestmögliche Maschinen-Startzeitpunkt $r_m(o_{j,k})$ berechnet. Dieser ergibt sich als Maximum aus der aktuellen Last der benötigten Maschine zuzüglich einer eventuell anfallenden Rüstzeit $s_{\lambda(m_{j,k}),j}$ und der Bereitschaftszeit $r(o_{j,k})$ der Operation selbst. Darauf aufbauend wird der minimale Endzeitpunkt C^* über alle einplanbaren Operationen bestimmt. Die Maschine, auf der dieser Wert erreicht wird, wird als kritische Maschine m^* definiert.

Anschließend wird die Konfliktmenge K gebildet. Diese enthält alle noch nicht verplanten Operationen auf der kritischen Maschine m^* , deren frühestmöglicher Startzeitpunkt echt kleiner als C^* ist. Aus dieser Menge wird mithilfe einer Prioritätsregel π eine Operation $o' = o_{j',k'}$ zur festen Einplanung ausgewählt.

Das Einplanen erfolgt durch das Setzen der Startzeit $t(o')$ auf den frühestmöglichen Maschinen-Startzeitpunkt $r_m(o')$ und der Berechnung des Fertigstellungszeitpunkts $C(o')$. Daraufhin werden die Last L_{m^*} der kritischen Maschine auf $C(o')$ aktualisiert und der Job-Index j' als der zuletzt auf m^* verarbeitete Job gespeichert. Falls eine Nachfolgeoperation $o_{j',k'+1}$ im selben Job existiert, wird deren Bereitschaftszeit $r(o_{j',k'+1})$ auf die Endzeit $C(o')$ angehoben, womit sie in der nächsten Iteration einplanbar wird. Abschließend wird die geplante Operation o' aus der Menge U entfernt. Der Algorithmus terminiert, sobald alle Operationen eingeplant wurden und somit $U = \emptyset$ gilt.

Da in jeder Iteration des Algorithmus Konflikte zwischen den einplanbaren Operationen auf derselben Maschine auftreten können, werden Prioritätsregeln eingesetzt, um diese Konflikte aufzulösen. Die Wahl der Prioritätsregel hat dabei einen maßgeblichen Einfluss auf den initialen Makespan, wie in Kapitel 4.1 aufgezeigt wird.

Zu Beginn wurden acht verschiedene Prioritätsregeln für das Problem untersucht. Zunächst vier einfache Regeln, wie sie unter anderem von Haupt (1988) [2] beschrieben werden. Sie beziehen sich meist auf eine einzelne Eigenschaft einer Operation.

1. **Shortest Processing Time (SPT)**: Sortiert die Operationen nach aufsteigender Prozessdauer.
2. **Shortest Processing Time + Setup (SPTS)**: Betrachtet zusätzlich zur Prozessdauer

er die Rüstzeit.

3. **Longest Processing Time (LPT)**: Sortiert Operationen nach absteigender Prozessdauer.
4. **First Come First Served (FCFS)**: Ordnet die Operationen der Reihenfolge nach.

Anschließend wurden vier weitere Prioritätsregeln nach Artigues et al. (2005) [3] und Kress et al. (2019) [4] implementiert.

5. **Minimum Start Time (MINSTART)**: Wählt die Operation $v = o_{j,k} \in K$ mit dem frühestmöglichen Startzeitpunkt $r_m(v)$ aus:

$$o' = \arg \min_{v=o_{j,k} \in K} (r_m(v)) \quad (3.1)$$

Sie minimiert Leerzeiten der Maschinen zu Beginn des Ablaufplans. Dies geschieht durch ein frühestmögliches Einplanen der Operationen. Als Nachteil wird die Bearbeitungszeit der gewählten Operationen ignoriert. Auch die Restarbeit des Jobs bleibt unberücksichtigt. Dies kann lange Jobs am Ende des Ablaufplans verzögern. Zudem werden Rüstzeiten nicht direkt minimiert.

6. **Minimum End Time (MINEND)**: Wählt die Operation $v = o_{j,k} \in K$ aus, welche den frühestmöglichen Fertigstellungszeitpunkt (Summe aus dem Startzeitpunkt $r_m(v)$ und der Bearbeitungszeit $p_{j,k}$) aufweist:

$$o' = \arg \min_{v=o_{j,k} \in K} (r_m(v) + p_{j,k}) \quad (3.2)$$

Diese Regel ist stark verwandt mit den Regeln MINSTART und SPT. Sie zielt jedoch auf den frühestmöglichen Abschluss einzelner Operationen ab und berücksichtigt zusätzlich die Maschinenverfügbarkeit. Dadurch werden belegte Maschinen schnell wieder freigegeben. Die Nachfolgeoperationen können somit früher beginnen und sehr lange Operationen werden zu Beginn vermieden, falls sie kürzere Operationen blockieren würden. Der Nachteil liegt im Fehlen einer globalen Sicht auf die Restarbeit der Jobs. Es können erhebliche Engpässe gegen Ende des Ablaufplans entstehen, die den Makespan negativ beeinflussen.

7. **Setup - Most Work Remaining (Setup - MWKR)**: Bevorzugt Operationen mit einer großen verbleibenden Restarbeit des zugehörigen Jobs $\sum_{l=k}^{q_j} p_{j,l}$, während gleichzeitig hohe sequenzabhängige Rüstzeiten $s_{u,v}$ zum vorherigen Job auf der Maschine bestraft werden:

$$o' = \arg \min_{v=o_{j,k} \in K} \left(s_{u,v} - \sum_{l=k}^{q_j} p_{j,l} \right) \quad (3.3)$$

wobei u die auf der Maschine m^* direkt zuvor verarbeitete Operation darstellt.

Die Regel bevorzugt Jobs mit großer Restarbeit. Gleichzeitig wirken hohe Rüstzeiten entgegengesetzt und machen die Operation unattraktiver. Dies vermeidet Bottlenecks gegen Ende des Ablaufs. Zudem wird der gesamte Prozess und nicht nur die unmittelbar nächste Operation betrachtet. Ein Nachteil ist, dass die Regel die Maschinenverfügbarkeit und Operationsdauer nicht mit betrachtet.

8. **Flow Due Date + Most Work Remaining + Shortest Setup (FDD + MWKR + SS)**: Diese kombinierte Heuristik minimiert das Verhältnis aus der bereits erbrachten Job-Arbeit $\sum_{l=1}^k p_{j,l}$ und der verbleibenden Restarbeit $\sum_{l=k}^{q_j} p_{j,l}$, addiert um die Rüstzeit $s_{u,v}$ zur direkt vorherigen Operation u auf derselben Maschine m^* :

$$o' = \arg \min_{v=o_{j,k} \in K} \left(\frac{\sum_{l=1}^k p_{j,l}}{\sum_{l=k}^{q_j} p_{j,l}} + s_{u,v} \right) \quad (3.4)$$

wobei u die auf der kritischen Maschine m^* direkt zuvor verarbeitete Operation darstellt.

Sie wägt den relativen Jobfortschritt und die Rüstzeit gegeneinander ab. Dies sorgt für eine ausgeglichene Maschinenauslastung. Jedoch dominiert bei geringen Rüstzeiten der relative Jobfortschritt und bei großen Rüstzeiten überwiegt diese das Fortschrittsverhältnis.

3.2 Metaheuristik - Tabu-Suche

Algorithm 2 Tabu Search für das Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten

```

1:  $S \leftarrow \hat{S}$ 
2:  $S^* \leftarrow S$ 
3:  $TL \leftarrow \emptyset$ 
4:  $I_{\text{total}} \leftarrow 0$ 
5:  $I \leftarrow 0$ 
6:  $\tau \leftarrow 0$ 
7: while  $I_{\text{total}} < I_{\text{total}}^{\max}$  and  $I < I^{\max}$  and  $\tau < \tau^{\max}$  do
8:    $I_{\text{total}} \leftarrow I_{\text{total}} + 1$ 
9:   Erzeuge die Nachbarschaft  $NB(S)$  der aktuellen Lösung
10:  if  $I \geq I^{\text{div}}$  then
11:    Ergänze zufällige nicht-kritische Bewegungen zu  $NB(S)$ 
12:  end if
13:  for  $S' \in NB(S)$  do (jede Nachbarlösung)
14:    if  $S'$  ist unzulässig or (tabuiert and Aspiration nicht erfüllt) then
15:      Entferne  $S'$  aus  $NB(S)$ 
16:    end if
17:  end for
18:  Wähle die beste zulässige Nachbarlösung  $S' \in NB(S)$ 
19:   $S \leftarrow S'$ 
20:  Entferne abgelaufene Einträge aus  $TL$ 
21:   $TL \leftarrow TL \cup \{\text{inverse Bewegung von } S'\}$ 
22:  if  $F(S) < F(S^*)$  then
23:     $S^* \leftarrow S$ 
24:     $I \leftarrow 0$ 
25:  else
26:     $I \leftarrow I + 1$ 
27:  end if
28: end while
29: return  $S^*$ 

```

Die Suche startet mit einer zulässigen Startlösung S . Die bisher beste Lösung wird mit $S^* \leftarrow S$ initialisiert. Zusätzlich werden Tabu-Liste TL , Gesamtiterationszähler I_{total} und Verbesserungszähler I initialisiert. Die Abbruchbedingungen sind maximale Gesamtiterationen, maximale Iterationen ohne Verbesserung und ein optionales Zeitlimit.

Die Nachbarschaft $NB(S)$ wird in jeder Iteration aus einer als Parameter übergebenen Nach-

barschaftsdefinition erzeugt. Zur Auswahl stehen Adjacent Swap in kritischen Blöcken, N5 und N6. Die gewählte Definition bestimmt Größe und Struktur der Kandidatenmenge.

Eine Nachbarlösung wird vor der Bewertung auf Zulässigkeit geprüft. Unzulässig sind Kandidaten, deren Bewegung im Dispositionsgraphen einen Zyklus erzeugt. Diese Kandidaten werden sofort verworfen.

Die Tabu-Tenure ist fest auf $T_{\text{tabu}} = 20$ Iterationen gesetzt. Beim Eintrag einer inversen Bewegung in TL wird die Verbotsdauer als Ablaufiteration gespeichert. Abgelaufene Einträge werden aus Effizienzgründen nicht in jeder Iteration entfernt, sondern periodisch alle 32 Iterationen bereinigt.

Vor der Auswahl wird die gesamte Kandidatenliste $NB(S)$ zufällig permutiert. Anschließend wird die erste beste gefundene zulässige Nachbarlösung übernommen. Dadurch entsteht bei mehreren gleichwertigen Kandidaten eine zufällige Tie-Break-Regel.

Tabuierte Bewegungen werden nur akzeptiert, wenn das Aspirationskriterium erfüllt ist. Die Bedingung lautet $F(S') < F(S^*)$. Damit bezieht sich die Aspiration auf die bisher beste Lösung und nicht auf die aktuelle Lösung.

Bleibt nach Zulässigkeits- und Tabu-Filterung kein Kandidat übrig, wird einmalig eine Menge zufälliger nicht-kritischer Bewegungen erzeugt. Dieser Fallback erfolgt unabhängig von I^{div} . Die zusätzlichen Kandidaten werden erneut auf Zulässigkeit und Tabu-Status geprüft. Erst wenn auch dann keine zulässige Bewegung existiert, wird die Suche beendet.

Wird eine Nachbarlösung S' akzeptiert, wird $S \leftarrow S'$ gesetzt und die inverse Bewegung in TL eingetragen. Falls $F(S) < F(S^*)$ gilt, wird $S^* \leftarrow S$ gesetzt und $I \leftarrow 0$. Andernfalls wird $I \leftarrow I + 1$ gesetzt. Nach Abschluss der Suche wird S^* zurückgegeben.

3.2.1 Kritischer Pfad und kritische Blöcke

Die Bewertung einer Lösung erfolgt auf Basis eines disjunktiven Graphen. Eine solche Modellierung kann auf die Arbeiten von Balas (1969) [5] zurückgeführt werden. Jede Operation wird durch einen Knoten repräsentiert. Zwischen den Knoten existieren konjunktive Kanten für die technologische Reihenfolge innerhalb eines Jobs sowie disjunktive Kanten für die Bearbeitungsreihenfolge auf einer Maschine. Die konjunktiven Kanten sind durch die Jobs der Instanz vorgegeben. Wohingegen die Richtung der disjunktiven Kanten zunächst offen und erst durch die in der aktuellen Lösung festgelegten Maschinenreihenfolgen bestimmt wird. Diese Kanten tragen die Bearbeitungsdauer und die zusätzlichen sequenzabhängige Rüstzeiten. Sobald für jede Maschine eine zulässige Reihenfolge vorliegt, ist der resultierende Graph azyklisch.

In dem azyklischen Graphen entspricht die Differenz aus frühester Startzeit einer Operation und letztem Fertigstellungszeitpunkts der Länge des längsten Pfades. Um diesen berechnen zu

können wird zunächst für jede Operation der Eingangsgrad bestimmt. Eine Operation besitzt höchstens zwei unmittelbare Vorgänger: einen Job-Vorgänger und einen Maschinen-Vorgänger. Daraufhin werden alle Operationen ohne Vorgänger mit Startzeit null in eine Warteschlange eingefügt. Bei der Verarbeitung einer Operation u werden ihre Nachfolger entlang beider Kantentypen relaxiert. Für jeden Nachfolger v gilt

$$start(v) = \max(start(v), completion(u) + s_{uv}),$$

wobei s_{uv} die zusätzliche Rüstzeit auf der betrachteten Maschinenkante bezeichnet und auf konjunktiven Kanten null ist. Die Maximumbildung stellt sicher, dass eine Operation erst startet, wenn alle unmittelbaren Vorgänger abgeschlossen sind. Erhöht eine Relaxation den Startzeitwert eines Nachfolgers, wird der auslösende Vorgänger als bindender Vorgänger gespeichert.

Werden im Verlauf der topologischen Verarbeitung nicht alle Operationen besucht, enthält der disjunktive Graph einen Zyklus. Eine solche Reihenfolge ist unzulässig und wird verworfen. Nach Abschluss der Berechnung ergibt sich der Makespan $C_{\max}$ als maximale Fertigstellungszeit über alle Operationen. Der kritische Pfad wird anschließend durch Rückverfolgung der gespeicherten bindenden Vorgänger konstruiert. Ausgehend von einer Operation mit Fertigstellungszeit $C_{\max}$ wird iterativ zum jeweiligen Vorgänger zurückgegangen, bis eine Quelle erreicht ist. Die umgekehrte Folge dieser Operationen bildet einen kritischen Pfad. Die Summe seiner Kantengewichte entspricht exakt dem Makespan.

Auf dieser Grundlage werden kritische Blöcke bestimmt. Ein kritischer Block ist eine maximale Teilfolge des kritischen Pfades, deren Operationen auf derselben Maschine liegen und dort unmittelbar aufeinanderfolgen. Diese Blöcke definieren den Suchraum der kritischkeitsbasierten Nachbarschaften [6].

Die aktuelle Implementierung liefert bei mehreren gleich langen kritischen Pfaden nur einen einzelnen Pfad zurück. Dies folgt erstens aus der Wahl des Endknotens, da bei mehreren Operationen mit identischer maximaler Fertigstellungszeit nur der erste gefundene Endpunkt verwendet wird. Zweitens wird bei Gleichstand konkurrierender Vorgänger nur die zuerst verarbeitete bindende Kante gespeichert. Der berechnete Makespan bleibt dadurch korrekt. Eingeschränkt ist jedoch die nachgelagerte Nachbarschaftserzeugung, weil kritische Blöcke ausschließlich aus diesem einen Pfad extrahiert werden. Existieren mehrere voneinander unabhängige kritische Pfade, können erzeugte Bewegungen einen sichtbaren Pfad verändern, ohne den Makespan zu reduzieren, weil ein weiterer unbeachteter Pfad weiterhin bindend bleibt. Im vorliegenden Verfahren wird dieser Effekt nur teilweise abgeschwächt. Zusätzliche nicht-kritische Diversifikationszüge können nach längerer Stagnation indirekt einen weiteren kritischen Pfad treffen. Außerdem kann sich durch eine geänderte Reihenfolge in einer späteren Iteration ein anderer gleich langer Pfad als kritisch manifestieren. Eine vollständige Behand-

lung würde die gleichzeitige Identifikation aller Endoperationen mit Fertigstellungszeit $C_{\max}$ und die Berücksichtigung mehrerer kritischer Pfade in der Blockbildung erfordern.

3.2.2 Nachbarschaftsdefinitionen

Für die Tabu-Suche werden drei Nachbarschaftsdefinitionen verwendet, die auf dem kritischen Pfad basieren. Der kritische Pfad ist die Folge von Operationen, die den Makespan $C_{\max}$ bestimmt. Eine Verzögerung einer Operation auf diesem Pfad erhöht unmittelbar die Gesamtdauer [7].

Zur Erzeugung von Bewegungen wird der kritische Pfad in kritische Blöcke zerlegt. Ein kritischer Block ist eine maximale Teilfolge von Operationen, die auf dem kritischen Pfad liegen und auf derselben Maschine unmittelbar aufeinanderfolgen. Die Funktion `critical_blocks()` durchläuft den kritischen Pfad und fasst Operationen so lange zu einem Block zusammen, wie Maschinenzuordnung und unmittelbare Nachbarschaft erhalten bleiben. Bei Maschinenwechsel oder fehlender Adjazenz wird ein neuer Block begonnen. Blöcke mit weniger als zwei Operationen werden verworfen, da kein sinnvoller Tausch definiert ist [6].

Die Einschränkung auf Operationen innerhalb kritischer Blöcke folgt der Eigenschaft, dass Verbesserungen des Makespans nur durch Änderungen innerhalb kritischer Blöcke erreicht werden. Vertauschungen über Blockgrenzen hinweg reduzieren den Makespan nicht [8].

Die Implementierung umfasst drei Nachbarschaften.

Adjacent-Swap-Nachbarschaft. Für jeden kritischen Block werden alle benachbarten Operationspaare vertauscht. Ein Block $[A, B, C, D]$ erzeugt die Züge (A, B) , (B, C) und (C, D) . Diese Variante betrachtet alle lokalen Adjazenzvertauschungen innerhalb kritischer Blöcke [8].

N5-Nachbarschaft. Pro kritischem Block werden nur die ersten beiden sowie die letzten beiden Operationen vertauscht. Ein Block $[A, B, C, D, E]$ erzeugt damit ausschließlich die Züge (A, B) und (D, E) . Die Reduktion basiert auf dem Ergebnis, dass Randvertauschungen makespanwirksam sein können, während innere Vertauschungen diese Eigenschaft nicht besitzen [6].

N6-Nachbarschaft. N6 erweitert N5 um Insert-Bewegungen. Zusätzlich zu den N5-Randvertauschungen kann jede innere Operation eines Blocks an den Blockanfang oder an das Blockende verschoben werden. Für den Block $[A, B, C, D, E]$ entstehen neben (A, B) und (D, E) beispielsweise die Sequenzen $[C, A, B, D, E]$ und $[A, C, D, E, B]$. Duplikate werden ausgeschlossen, wenn ein Insert-Zug äquivalent zu einem bereits enthaltenen N5-Zug ist. Diese Erweiterung vergrößert die strukturelle Diversität der Nachbarschaft bei weiterhin begrenzter Kandidatenzahl [9].

3.3 Hard- und Softwarespezifikationen

Die Instanzen wurden auf einem Apple MacBook Pro aus dem Jahr 2023 ausgeführt. Das System ist mit einem Apple M3 Pro Prozessor sowie 18 GB gemeinsam genutztem Arbeitsspeicher ausgestattet. Die Berechnungen wurden lokal auf dem Gerät durchgeführt, ohne zusätzliche externe Rechenressourcen oder Serverinfrastruktur zu verwenden.

Die algorithmische Umsetzung erfolgte in der Programmiersprache Python in der Version 3.13.11 Als Betriebssystem kam macOS Tahoe 26.5.1 zum Einsatz. Für die exakte Validierung wurde der CP-Solver aus den Google OR-Tools in der Version 9.15.6755 verwendet. Die Erstellung der Gantt-Charts zur Visualisierung der Ablaufpläne erfolgte mithilfe von Google Charts.

Kapitel 4

Ergebnisse

In diesem Kapitel werden die Ergebnisse der entwickelten Heuristiken quantitativ ausgewertet und analysiert. Als Testdatenbasis dienen hierbei die ClassroomSet-Instanzen, welche sich aus zehn Sets mit jeweils drei Instanzen zusammensetzen. Die Evaluation gliedert sich in drei wesentliche Abschnitte. Zunächst wird die Leistung der verschiedenen Prioritätsregeln innerhalb des Giffler-Thompson-Algorithmus verglichen, um eine geeignete Startlösung zu bestimmen. Anschließend erfolgt die Auswertung der Tabu-Suche, bei der die drei Nachbarschaftsstrukturen Adjacent-Swap, N5 und N6 gegenübergestellt werden. Den Abschluss bildet die Validierung der Metaheuristik durch den Vergleich mit den Ergebnissen des exakten CP-Solvers der Google OR-Tools.

4.1 Eröffnungsheuristik Ergebnisse

Zur Generierung einer zulässigen Startlösung für das Job-Shop-Scheduling-Problem wurden alle acht in Abschnitt 3.1 beschriebenen Prioritätsregeln auf die ClassroomSet-Instanzen angewendet. Die Wahl der jeweiligen Prioritätsregel steuert dabei die Konfliktauflösung auf den Maschinen und beeinflusst somit direkt den resultierenden Makespan. Die Auswertung dient dem systematischen Vergleich der Heuristiken hinsichtlich ihrer Lösungsqualität. Dabei sollen die am besten geeignete Regel zur Erzeugung der Startlösung $\hat{S}$ für die nachgelagerte Tabu-Suche identifiziert werden.

In Abbildung 1 werden die Ergebnisse aller acht implementierten Prioritätsregeln über ausgewählte Instanzen hinweg verglichen.

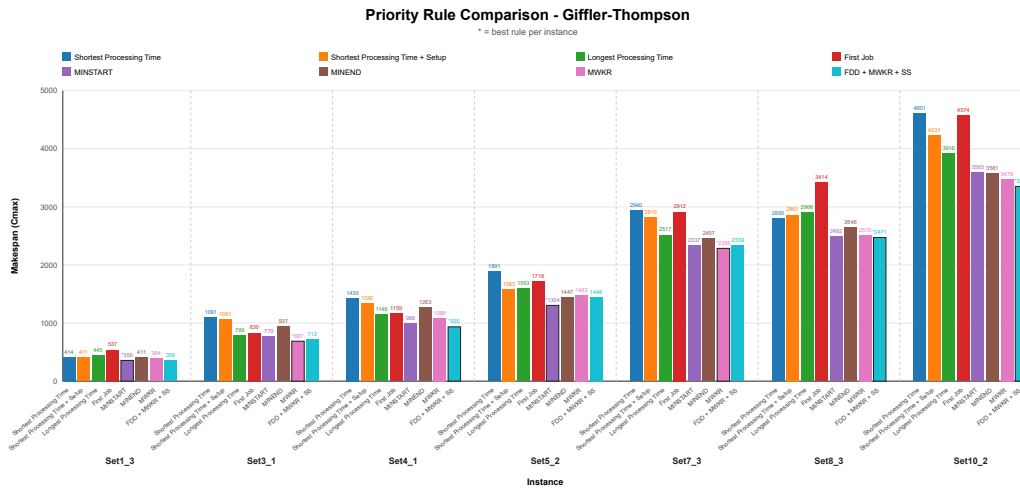


Abbildung 1: Vergleich der prioritätsbasierten Eröffnungsheuristiken anhand des erreichten Makespans $C_{\max}$ für ausgewählte Classroom Instanzen.

Es wird schnell deutlich, dass die trivialen Prioritätsregeln (SPT, SPTS, LPT, FCFS) im Durchschnitt deutlich schlechtere Ergebnisse liefern als die vier komplexeren Regeln. In den dargestellten Ergebnissen konnte keine der einfacheren Regeln die beste Lösung generieren.

Zur besseren Veranschaulichung der Unterschiede werden in der folgenden Tabelle noch einmal die Ergebnisse aller acht Prioritätsregeln über die gesamte ClassroomSet-Datenbasis hinweg gegenübergestellt.

Die Bewertung erfolgt anhand von fünf Leistungskennzahlen. Der mittlere Makespan $C_{\max}$ gibt die durchschnittliche Gesamtlaufzeit über alle Instanzen an. Die mittlere relative prozentuale Abweichung (RPD) beschreibt die durchschnittliche Abweichung zur jeweils besten Heuristiklösung. Die maximale RPD zeigt die größte Abweichung bei einer einzelnen Instanz. Die Spalte Bestlösungen zählt, wie oft eine Regel die beste Startlösung ermittelt hat. Der mittlere Rang gibt die durchschnittliche Platzierung der Regel im direkten Vergleich aller Heuristiken an.

Tabelle 3: Kennzahlen Vergleich der Giffier-Thompson-Prioritätsregeln

Prioritätsregel	Mittlerer $C_{\max}$	Mittlere RPD (%)	Maximale RPD (%)	Bestlösungen	Mittlerer Rang
SPT	2103,90	27,97	58,81	0	6,67
SPTS	2068,47	25,38	54,44	0	6,20
LPT	1988,93	19,91	39,29	0	5,43
FCFS	2161,73	27,88	50,84	0	6,53
MINSTART	1715,80	2,90	15,20	13	1,83
MINEND	1869,07	13,73	39,24	2	3,70
Setup - MWKR	1730,07	5,22	19,17	10	2,30
FDD + MWKR + SS	1805,07	7,68	30,90	5	2,87

Die Tabelle bestätigt die Überlegenheit der komplexeren Heuristiken. Die Regel MINSTART

erzielt über alle 30 Instanzen hinweg die besten Ergebnisse. Sie weist den geringsten mittleren Makespan von 1715,80 auf. Zudem erreicht sie mit 2,90 % die kleinste mittlere relative prozentuale Abweichung (RPD). Auf 13 der 30 Instanzen liefert MINSTART die beste Startlösung. Ihr mittlerer Rang liegt damit bei 1,83.

Dicht darauf folgt die Regel Setup - MWKR. Sie erzielt einen mittleren Makespan von 1730,07. In zehn Fällen liefert sie die beste Startlösung und erreicht damit einen mittleren Rang von 2,30. Die Regel FDD + MWKR + SS belegt den dritten Platz. Sie liefert fünf Bestlösungen und einen mittleren Rang von 2,87. Die Regel MINEND liegt deutlich zurück und erreicht mit zwei Bestlösungen einen mittleren Rang von 3,70.

Die einfachen Prioritätsregeln zeigen eine deutlich schwächere Performance. Keine dieser Regeln kann eine Bestlösung erzeugen. LPT schneidet mit einem mittleren Makespan von 1988,93 noch am besten ab. SPT und FCFS bilden mit Rängen über 6,5 die Schlusslichter. Die Hinzunahme von Rüstzeiten bei SPTS führt zu einer leichten Verbesserung gegenüber SPT. Der mittlere Makespan sinkt dabei von 2103,90 auf 2068,47. Vergleichsweise ist diese Verbesserung jedoch zu vernachlässigen.

4.2 Metaheuristik Ergebnisse

Nachdem eine geeignete Startlösung durch die Eröffnungsheuristik bestimmt wurde, wird in diesem Abschnitt die Leistungsfähigkeit der entwickelten Tabu-Suche evaluiert. Im Fokus steht dabei der Vergleich der drei Nachbarschaftsstrukturen *Adjacent-Swap*, *N5* und *N6*.

Die Untersuchung erfolgt zunächst exemplarisch anhand ausgewählter Einzelinstanzen. Dazu wird das Konvergenzverhalten grafisch für zwei kleinere Instanzen (4_2 und 4_3) sowie für eine größere Instanz (10_1) unter verschiedenen Zeitlimits (90 und 300 Sekunden) analysiert. Daran anschließend werden die Ergebnisse über alle Instanzen hinweg aggregiert dargestellt. Somit wird eine fundierte Aussage über die Gesamtpformance der Nachbarschaften ermöglicht. Das primäre Ziel der ersten grafischen Darstellung ist es zu analysieren, wie effektiv die jeweiligen Nachbarschaften den Suchraum explorieren und inwieweit sie in der Lage sind, die Startlösung zu verbessern.

Restriktive Nachbarschaften wie *N5* und *N6* intensivieren die Suche durch die ausschließliche Betrachtung bestimmter Operationen auf dem kritischen Pfad. Hierbei besteht jedoch das Risiko, dass sich die Suche mangels zulässiger Alternativen in lokalen Optima festläuft. Die breitere *Adjacent-Swap*-Nachbarschaft gestattet hingegen auch Vertauschungen von direkt benachbarten Operationen innerhalb eines kritischen Blocks. Diese Züge sind bei *N5* und *N6* ausgeschlossen. Dadurch vergrößert sich die Anzahl zulässiger Übergänge, was alternative Pfade zur Überwindung lokaler Optima bereitstellt.

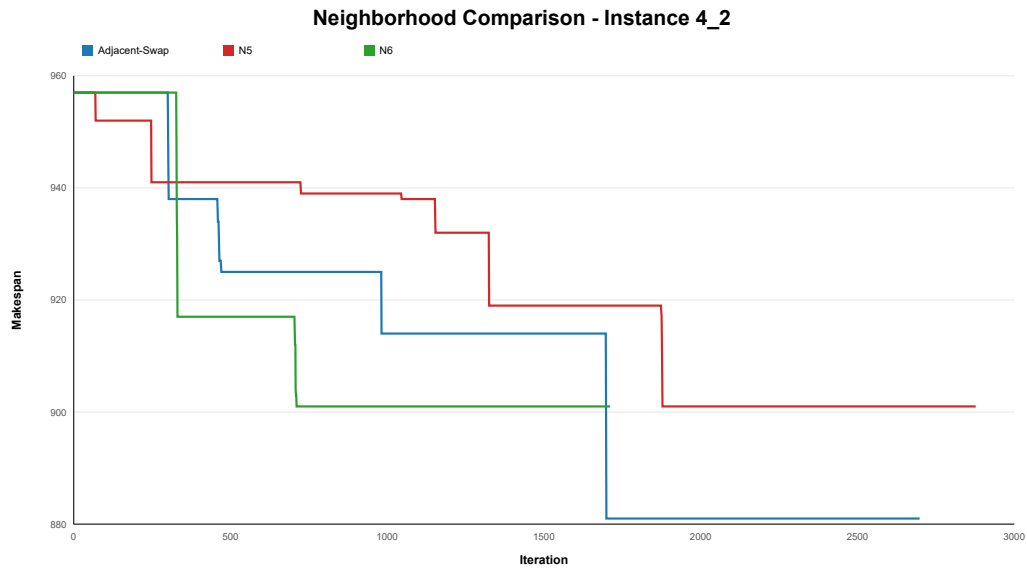


Abbildung 2: Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 4_2.

Für die Instanz 4_2 starten alle drei Nachbarschaften mit einem identischen initialen Makespan von 957. Zu Beginn der Suche findet die *N6*-Nachbarschaft am schnellsten eine Verbesserung und erreicht bereits nach 711 Iterationen ihren minimalen Makespan von 901. Die *Adjacent-Swap*-Nachbarschaft verbessert sich anfangs etwas langsamer. Sie kann jedoch im weiteren Verlauf lokale Optima überwinden. Nach 1698 Iterationen konvergiert sie schließlich bei einem deutlich besseren Makespan von 881. Die *N5*-Nachbarschaft verhält sich ähnlich wie *N6*, erreicht jedoch erst nach 1877 Iterationen den gleichen Makespan von 901.

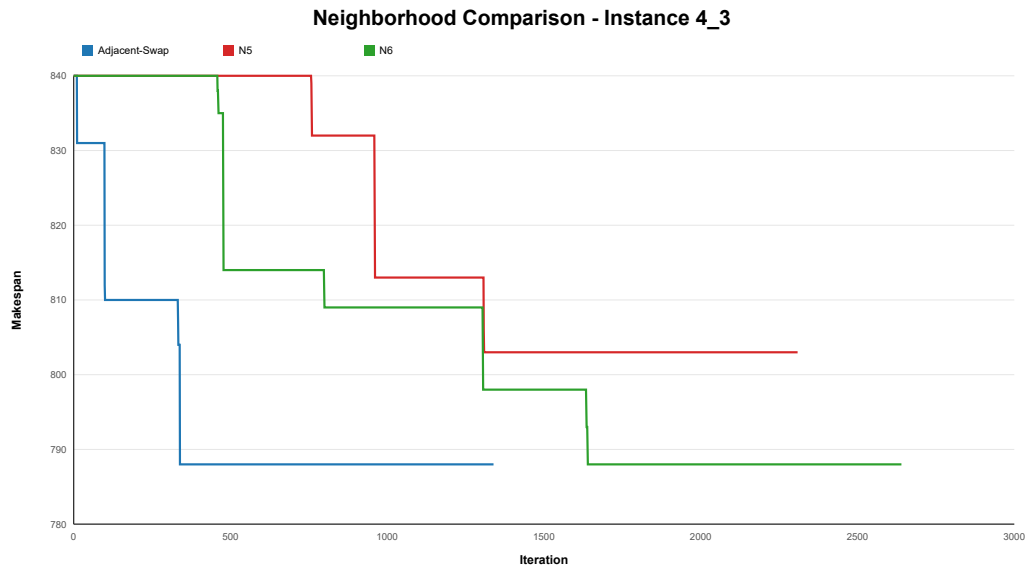


Abbildung 3: Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 4_3.

Ein ähnliches Verhalten zeigt sich bei der Instanz 4_3, bei der die Startlösung einen Makespan von 840 aufweist. Hier erzielen sowohl die *Adjacent-Swap*- als auch die *N6*-Nachbarschaft den optimalen Makespan von 788. *Adjacent-Swap* konvergiert dabei bereits nach 339 Iterationen. Die *N6*-Nachbarschaft benötigt hingegen 1640 Iterationen für dasselbe Ergebnis. Die *N5*-Nachbarschaft fällt mit einem minimalen Makespan von 803 deutlich zurück, welcher nach 1309 Iterationen erreicht wird.

In den folgenden zwei Abbildungen wird die sehr große Instanzen 10_1 mit unterschiedlichen Zeitlimits untersucht.

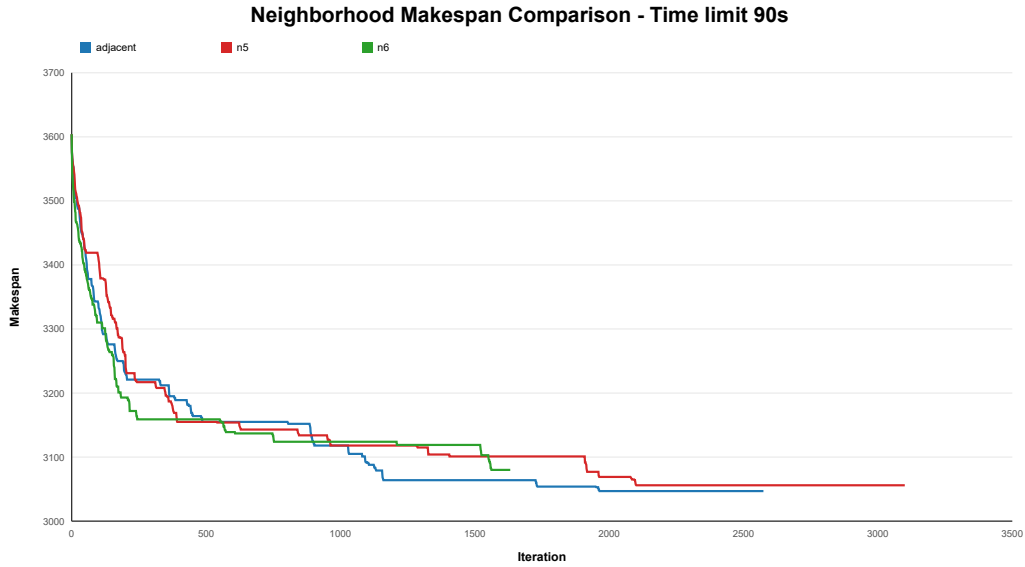


Abbildung 4: Vergleich des erreichten Makespan der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 10_1 mit einem Zeitlimit von 90 Sekunden.

Abbildung 4 zeigt den Verlauf der Suche bei einem Zeitlimit von 90 Sekunden. Zu Beginn der Suche sinkt der Makespan bei allen Nachbarschaften stark, wobei die selektive *N6*-Nachbarschaft in den ersten 500 Iterationen den schnellsten Fortschritt erzielt. Ab etwa Iteration 1000 erzielt die *Adjacent-Swap*-Nachbarschaft jedoch kontinuierlich bessere Ergebnisse als die Vergleichsverfahren. Sie wird aufgrund des strikten Zeitlimits bei Iteration 2660 mit einem Makespan von 3047 gestoppt. Auch die *N6*-Nachbarschaft wird aus dem selben Grund bei Iteration 1632 mit einem Makespan von 3080 abgebrochen. Die *N5*-Nachbarschaft dagegen wird aufgrund der maximalen Anzahl Iterationen ohne Verbesserung beendet und erreicht trotz 3100 Iterationen nur einen Makespan von 3056. Dies verdeutlicht, dass die breitere Nachbarschaft auch in kürzeren Zeitfenstern trotz unvollständiger Suche die höchste Lösungsqualität liefert.

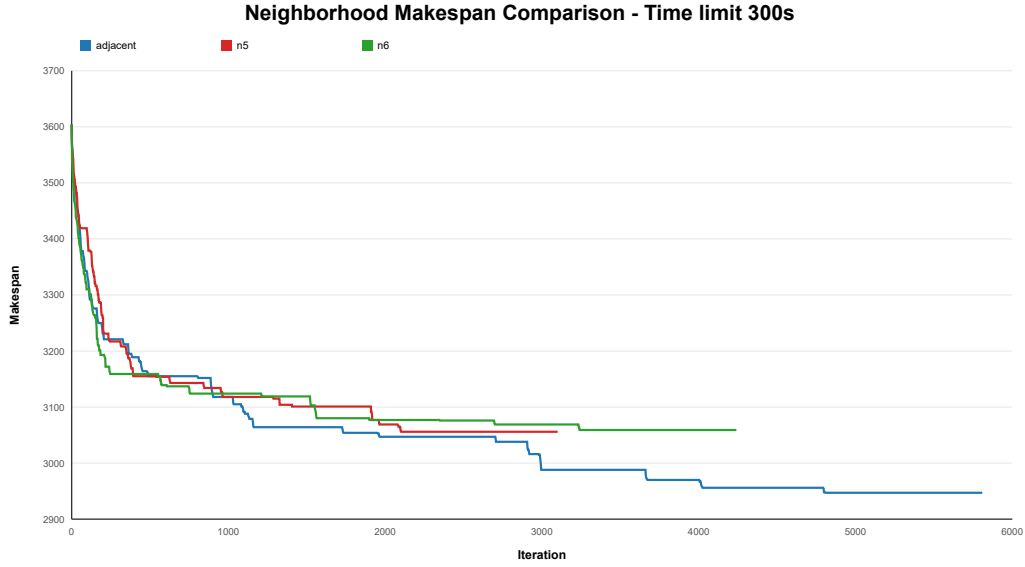


Abbildung 5: Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 10_1 mit einem Zeitlimit von 300 Sekunden.

Mit dem erweiterten Zeitlimit von 300 Sekunden wird das Konvergenzverhalten der Nachbarschaftsdefinitionen bei größeren Instanzen noch deutlicher.

Sowohl *N5* als auch *N6* entkommen nicht aus lokalen Optima und brechen die Suche aufgrund stagnierender Ergebnisse vorzeitig ab. Die *N5*-Nachbarschaft erreicht bereits bei Iteration 2100 ihre beste Lösung von 3056 und bricht die Suche nach 3100 Iterationen ab. Die *N6*-Nachbarschaft findet nach Iteration 3240 keine Verbesserung mehr und beendet die Suche bei Iteration 4240 mit einem Makespan von 3059. Die breitere *Adjacent-Swap*-Nachbarschaft hingegen überwindet lokale Optima fortlaufend und erzielt bei Iteration 4808 mit 2947 den besten Makespan. Alle drei Nachbarschaften brechen ihre Suche aufgrund fehlender Verbesserungen jedoch vor dem Zeitlimit ab.

Um eine noch umfassendere Bewertung der Ergebnisse zu ermöglichen, werden die Nachbarschaftsdefinitionen in der folgenden Tabelle noch einmal zusammenfassend über alle 30 gegebenen Instanzen evaluiert.

Tabelle 4: Vergleich der Nachbarschaftsdefinitionen über alle 30 Classroom Instanzen mit einem Zeitlimit von 90 s.

Nachbarschaft	Ø Verbesserung	Gewonnene Inst.	Ø Gap	Ø Iterationen	Iterationen (Summe)	Ø Makespan	Ø Laufzeit
Adjacent	11,01 %	18/30	0,57 %	1957	58.697	1459,7	23,5 s
N5	9,86 %	10/30	1,90 %	2175	65.259	1475,8	17,5 s
N6	10,59 %	15/30	1,05 %	2530	75.893	1470,5	36,8 s

Die aggregierten Ergebnisse in Tabelle 4 bestätigen die Leistungsvorteile der *Adjacent-Swap*-

Nachbarschaft. Sie erzielt über alle 30 Instanzen hinweg mit durchschnittlich 11,01 % die größte Verbesserung gegenüber den GT-Startlösungen. Zudem erreicht sie mit 1459,7 den geringsten mittleren Makespan und weist mit 0,57 % den kleinsten durchschnittlichen Abstand zur jeweils besten Nachbarschaft auf. Mit 18 gewonnenen Instanzen liefert sie in den meisten Fällen das beste heuristische Ergebnis. Die Nachbarschaft *N6* folgt mit einer durchschnittlichen Verbesserung von 10,59 % und 15 gewonnenen Instanzen knapp dahinter. Sie benötigt jedoch mit einer mittleren Laufzeit von 36,8 s die meiste Rechenzeit unter den getesteten Verfahren. Die *N5*-Nachbarschaft schneidet mit einer Verbesserung von 9,86 % und einem Gap von 1,90 % in allen Qualitätsmetriken am schlechtesten ab. Sie weist jedoch aufgrund der schnellen Konvergenz mit 17,5 s die kürzeste mittlere Laufzeit auf.

Zusammenfassend zeigt sich, dass die *Adjacent-Swap*-Nachbarschaft über alle getesteten Szenarien hinweg die besten und stabilsten Ergebnisse liefert. Obwohl die Nachbarschaften *N5* und *N6* durch die gezielte Veränderung von Operationen auf dem kritischen Pfad theoretisch effizienter sein sollten, erweist sich die einfachere Struktur von *Adjacent-Swap* in der Praxis als überlegen. Dies lässt sich darauf zurückführen, dass *Adjacent-Swap* weniger restriktiv ist und somit lokale Optima leichter überwinden kann, während die Berechnung einer einzelnen Iteration im Vergleich zu den komplexeren Nachbarschaften weniger Rechenzeit erfordert.

4.3 Vergleich CP-Solver

In diesem Abschnitt werden die Ergebnisse der Tabu-Suche mit dem exakten Constraint-Programming-Solver (CP-Solver) der Google OR-Tools verglichen. Dieser Vergleich dient als Maßstab für die Qualität der heuristischen Lösungen. Da der CP-Solver bei ausreichender Laufzeit optimale Lösungen garantieren kann, ermöglicht er die Bestimmung des verbleibenden Optimierungspotenzials der Heuristiken. Aufgrund der NP-Schwere des Job-Shop-Scheduling-Problems ist die Rechenzeit des Solvers in der Praxis jedoch limitiert. Daher wird evaluiert, wie sich die Verfahren unter identischen zeitlichen Restriktionen verhalten.

Abbildung 6 zeigt den Vergleich der Nachbarschaften mit dem CP-Solver bei einem Zeitlimit von 90 Sekunden auf dem Instanzset 10.

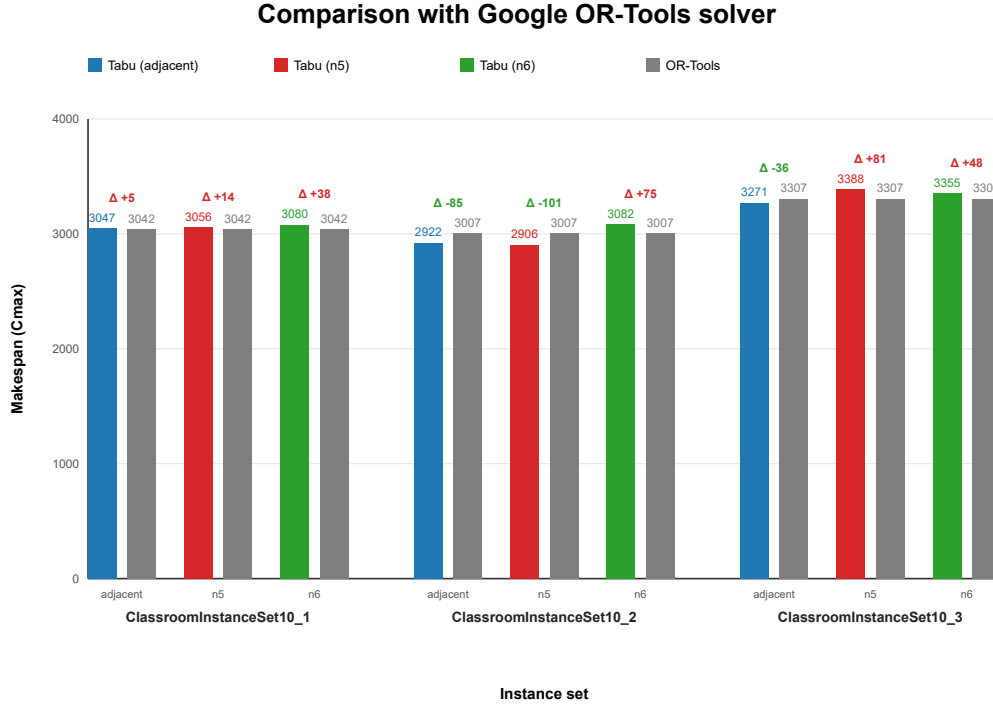


Abbildung 6: Vergleich der Nachbarschaftsdefinitionen mit dem Google OR-Tools Solver anhand des erreichten Makespans für Instanzset 10 mit einem Zeitlimit von 90 Sekunden.

Die Ergebnisse verdeutlichen, dass der CP-Solver unter dem strikten Zeitlimit von 90 Sekunden an seine Grenzen stößt. Bei der Instanz 10_1 erreicht der Solver einen Makespan von 3042, während die *Adjacent-Swap*-Nachbarschaft mit einem Abstand von nur +5 (+0,16 %) ein nahezu identisches Ergebnis von 3047 erzielt. Bei der Instanz 10_2 kann die *Adjacent-Swap*-Nachbarschaft den CP-Solver deutlich übertreffen und erzielt einen um 85 Punkte (-2,83 %) besseren Makespan von 2922. Überraschenderweise erzielt die sonst eher leistungsschwache Nachbarschaft *N5* bei dieser Instanz mit 2906 sogar ein um 101 Punkte besseres Ergebnis als der Solver, während *N6* mit 3082 abfällt. Auch bei der Instanz 10_3 übertrifft *Adjacent-Swap* (3271) den CP-Solver (3307) um 36 Punkte. Die beiden anderen Nachbarschaften liegen hier etwas über dem CP-Solver. Dies demonstriert, dass Heuristiken in stark begrenzten Zeitfenstern bei komplexeren Instanzen eine höhere Lösungsqualität finden können als ein exakter Solver.

Abbildung 7 stellt den gleichen Vergleich unter einem erweiterten Zeitlimit von 300 Sekunden dar.

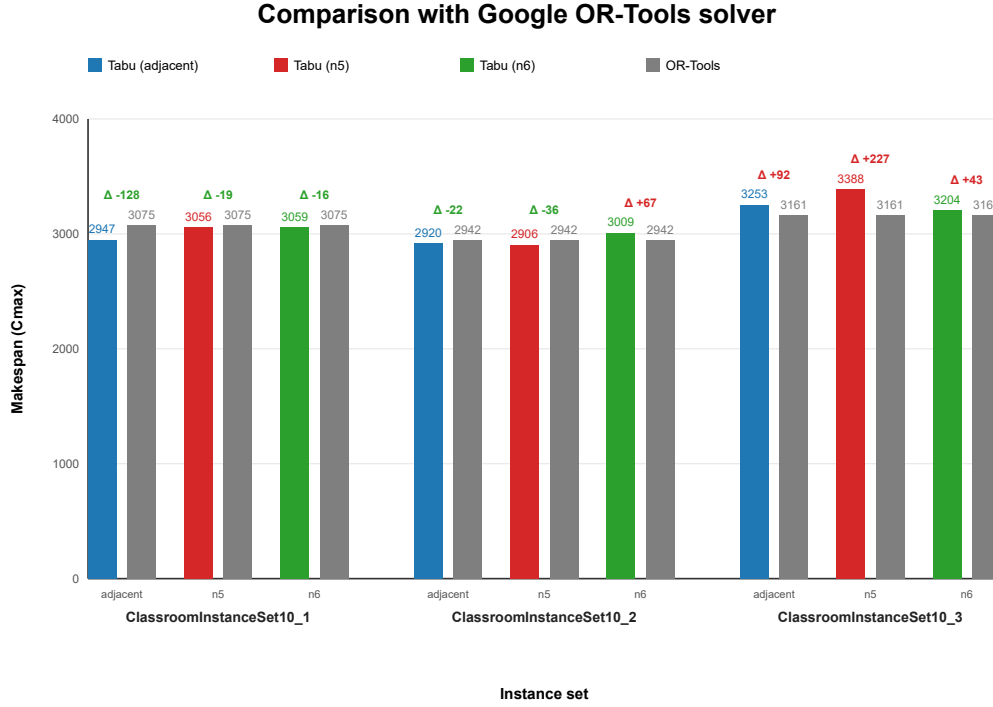


Abbildung 7: Vergleich der Nachbarschaftsdefinitionen mit dem Google OR-Tools Solver. Zeitlimit 300 Sekunden.

Mit der Erhöhung der Rechenzeit auf 300 Sekunden verbessert sich die Lösungsqualität über alle Heuristiken hinweg. Interessanterweise kann die *Adjacent-Swap*-Nachbarschaft hier bei der Instanz 10_1 den CP-Solver deutlich übertreffen und erreicht einen Makespan von 2947 im Vergleich zu 3075 des Solvers. Auch bei der Instanz 10_2 bleibt die Heuristik führend. *Adjacent-Swap* erzielt einen um 22 Punkte (-0,75 %) besseren Makespan (2920) als der CP-Solver (2942), während die Nachbarschaft *N5* sogar ein um 36 Punkte besseres Ergebnis erreicht. Lediglich bei der Instanz 10_3 bleibt der CP-Solver mit 3161 führend, wobei der Abstand von *N6* mit +43 Punkten moderat ausfällt. Die Nachbarschaften *N5* und *N6* liegen in den meisten Fällen hinter *Adjacent-Swap* zurück.

Um diesen Vergleich quantitativ über alle Instanzen abzurunden, werden die aggregierten Ergebnisse aller drei Nachbarschaften im Vergleich zum CP-Solver in Tabelle 5 zusammengefasst. Δ gibt dabei die mittlere Abweichung der jeweiligen Nachbarschaftsde vom CP-Solver an.

Tabelle 5: Vergleich der Tabu-Suche-Nachbarschaften mit dem CP-Solver über alle 30 Classroom Instanzen (Zeitlimit: 90 s).

Methode	$\varnothing C_{\max}$	$\varnothing$ Iter.	$\varnothing$ Zeit	$\varnothing \Delta$ abs.	$\varnothing \Delta$ rel.
Adjacent-Swap	1459.7	1957	23.5 s	+47.0	+2.95 %
N5	1475.8	2175	17.5 s	+63.1	+4.29 %
N6	1470.5	2530	36.8 s	+57.7	+3.41 %
CP-Solver (OR-Tools)	1412.8	—	47.3 s	—	—

Die *Adjacent-Swap*-Nachbarschaft erzielt mit einem mittleren $C_{\max}$ von 1459,7 und einer Abweichung von +2,95 % die besten heuristischen Ergebnisse. Sie benötigt im Mittel 23,5 s und führt durchschnittlich 1957 Iterationen durch. *N6* belegt mit +3,41 % den zweiten Platz, verursacht jedoch mit 36,8 s die höchste Rechenzeit, da die erweiterte Nachbarschaft pro Iteration aufwendiger zu evaluieren ist. *N5* schneidet mit einer mittleren Abweichung von +4,29 % am schwächsten ab, konvergiert dafür mit 17,5 s am schnellsten. Der CP-Solver erreicht einen mittleren Makespan von 1412,8. Für die kleineren Instanzsets (Sets 1–4) findet er in Sekundenbruchteilen optimale Lösungen, während er bei größeren Instanzen (Sets 6–10) innerhalb der 90-Sekunden-Schranke nur zulässige Lösungen liefert. Die Ergebnisse zeigen, dass die Tabu-Suche als Heuristik bei komplexen Instanzen unter identischer Zeitbeschränkung wettbewerbsfähig bleibt und in Einzelfällen den exakten Solver übertrifft.

Abbildungsverzeichnis

1	Vergleich der prioritätsbasierten Eröffnungsheuristiken anhand des erreichten Makespans $C_{\max}$ für ausgewählte Classroom Instanzen.	17
2	Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 4_2.	19
3	Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 4_3.	20
4	Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 10_1 mit einem Zeitlimit von 90 Sekunden.	21
5	Vergleich des erreichten Makespans der verschiedenen Nachbarschaftsdefinitionen anhand der Instanz 10_1 mit einem Zeitlimit von 300 Sekunden.	22
6	Vergleich der Nachbarschaftsdefinitionen mit dem Google OR-Tools Solver anhand des erreichten Makespans für Instanzset 10 mit einem Zeitlimit von 90 Sekunden.	24
7	Vergleich der Nachbarschaftsdefinitionen mit dem Google OR-Tools Solver. Zeitlimit 300 Sekunden.	25

Quellcodeverzeichnis

1	Giffler-Thompson-Algorithmus	7
2	Tabu Search für das Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten	11

Tabellenverzeichnis

1	Notation der Eröffnungsheuristik (Giffler-Thompson-Algorithmus)	5
2	Notation der Tabu-Search-Heuristik	6
3	Kennzahlen Vergleich der Giffler-Thompson-Prioritätsregeln	17
4	Vergleich der Nachbarschaftsdefinitionen über alle 30 Classroom Instanzen mit einem Zeitlimit von 90 s.	22
5	Vergleich der Tabu-Suche-Nachbarschaften mit dem CP-Solver über alle 30 Classroom Instanzen (Zeitlimit: 90 s).	26

Literaturverzeichnis

- [1] B. Giffler und G. L. Thompson, „Algorithms for solving production-scheduling problems,” *Operations Research*, Vol. 8, Nr. 4, S. 487–503, 1960. (Zitiert auf Seite 8.)
- [2] R. Haupt, „A Survey of Priority Rule-Based Scheduling,” *OR Spektrum*, Vol. 11, Nr. 1, S. 3–16, Mar. 1989. (Zitiert auf Seite 8.)
- [3] C. Artigues, P. Lopez, und P.-D. Ayache, „Schedule Generation Schemes for the Job-Shop Problem with Sequence-Dependent Setup Times: Dominance Properties and Computational Analysis,” Vol. 138, Nr. 1, S. 21–52. [Online]. Available: <http://link.springer.com/10.1007/s10479-005-2443-4> (Zitiert auf Seite 9.)
- [4] D. Kress, D. Müller, und J. Nossack, „A worker constrained flexible job shop scheduling problem with sequence-dependent setup times,” Vol. 41, Nr. 1, S. 179–217. [Online]. Available: <http://link.springer.com/10.1007/s00291-018-0537-z> (Zitiert auf Seite 9.)
- [5] E. Balas, „Machine Sequencing via Disjunctive Graphs: An Implicit Enumeration Algorithm,” *Operations Research*, Vol. 17, Nr. 6, S. 941–957, 1969. [Online]. Available: <http://www.jstor.org/stable/168317> (Zitiert auf Seite 12.)
- [6] E. Nowicki und C. Smutnicki, „A Fast Taboo Search Algorithm for the Job Shop Problem,” Vol. 42, Nr. 6, S. 797–813. [Online]. Available: <http://www.jstor.org/stable/2634595> (Zitiert auf Seiten 13 und 14.)
- [7] J. Blazewicz, K. H. Ecker, E. Pesch, G. Schmidt, M. Sterna, und J. Weglarz, *Handbook on Scheduling: From Theory to Practice*. Springer International Publishing. [Online]. Available: <http://link.springer.com/10.1007/978-3-319-99849-7> (Zitiert auf Seite 14.)
- [8] Peter J. M. van Laarhoven, Emile H. L. Aarts, und J. K. Lenstra, „Job Shop Scheduling by Simulated Annealing,” Vol. 40, Nr. 1, S. 113–125. [Online]. Available: <http://www.jstor.org/stable/171189> (Zitiert auf Seite 14.)
- [9] E. Balas und A. Vazacopoulos, „Guided Local Search with Shifting Bottleneck for Job Shop Scheduling,” *Management Science*, Vol. 44, Nr. 2, S. 262–275, 1998. [Online]. Available: <http://www.jstor.org/stable/2634500> (Zitiert auf Seite 14.)